

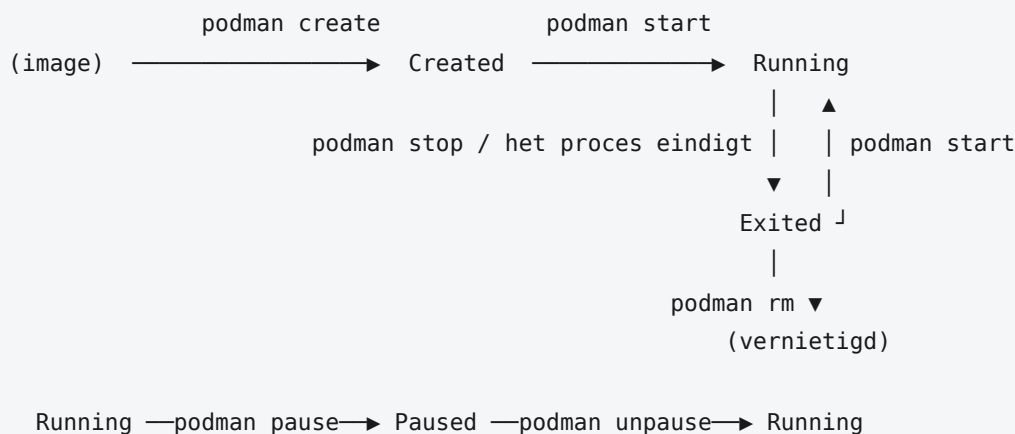
Lab 03 — De levenscyclus van een container

Theorie — geboorte, leven, signalen en dood van een container; waarom PID 1 alles verandert, en wie je containers herstart als er geen daemon is.

Doelstellingen

- De toestanden van een container kennen en de commando's die van de ene naar de andere leiden.
- Begrijpen waarom een container "vanzelf stopt".
- De relatie tussen het **hoofdproces**, de **signalen** en de **exitcode** beheersen.
- Kiezen tussen `exec` en `attach`, tussen voorgrond en achtergrond.
- Weten wat een *restart policy* werkelijk doet — en wat ze zonder daemon niet kan.

1. De toestanden



Commando	Effect
<code>podman create</code>	Bereidt de container voor (schrijflaag, config) zonder hem te starten
<code>podman start</code>	Start het hoofdproces
<code>podman run</code>	<code>create</code> + <code>start</code> (+ <code>pull</code> als de image ontbreekt)
<code>podman stop</code>	Vraagt beleefd om te stoppen, en forceert na een wachttijd
<code>podman kill</code>	Forceert de stop onmiddellijk
<code>podman restart</code>	<code>stop</code> en dan <code>start</code>
<code>podman pause</code>	Bevriest de processen (cgroup freezer), zonder ze te stoppen
<code>podman rm</code>	Vernietigt de container en zijn schrijflaag

Een `Exited`-container is niet dood: hij behoudt configuratie, schrijflaag en logs. Je kunt hem inspecteren, herstarten, er bestanden uit halen. Alleen `rm` vernietigt.

2. De basisregel: een container leeft zolang zijn PID 1

ONTHOUDEN

Een container stopt precies wanneer zijn **hoofdproces** eindigt. Niet ervoor, niet erna. Meer valt er niet te begrijpen.

Dat verklaart bijna elke "mijn container stopt vanzelf":

- `podman run alpine` stopt meteen — het standaardcommando `/bin/sh` leest zonder terminal niets en sluit af.
- `podman run nginx` blijft leven — nginx draait op de voorgrond en geeft nooit de hand terug.
- Een script dat een dienst op de **achtergrond** zet (`java -jar ... &`) sterft meteen: het script eindigt, dus PID 1 ook.

Vandaar een ontwerpregel: **een image start haar dienst op de voorgrond**. Geen daemon, `systemd` of `nohup` in een container: de engine is de dienstbeheerder.

→ LINUX

Op een gewone Linux-machine is PID 1 `init` (vandaag `systemd`): het eerste proces dat de kernel aanmaakt, voorouder van alle andere. De kernel behandelt het apart: sterft het, dan stopt het systeem; en het **negeert standaard signalen** waarvoor het geen handler installeerde, zodat een onhandige `kill` de machine niet neerhaalt. In een container erft *jouw applicatie* die `init`-status — met zijn privileges en valkuilen.

Gevolg: een container is gemaakt voor **één hoofdproces**. API en database in dezelfde container breekt het model — je kunt ze niet meer apart herstarten, bewaken of schalen.

3. Voorgrond, achtergrond, en het duo `-it`

```
podman run nginx           # voorgrond: de terminal is geblokkeerd, logs getoond
podman run -d nginx        # losgekoppeld: geeft de hand terug, toont de container-ID
podman run -it alpine sh    # interactief: je krijgt een bruikbare shell
podman run --rm alpine date # eenmalige uitvoering, container verwijderd bij afsluiten
```

`-it` wordt slecht begrepen omdat het **twee aparte opties** zijn: `-i` houdt de standaardinvoer open — zonder ziet een shell zijn `stdin` gesloten en stopt hij meteen; `-t` kent een pseudo-terminal toe — prompt, echo van de toetsen, `Ctrl+C`. In scripts of CI: `-i` alleen; voor mensen: `-it`.

VALKUIL

Op de voorgrond stuurt `Ctrl+C` `SIGINT` naar het proces van de container: nginx stopt. In `-it`-modus laat de reeks `Ctrl+P` gevolgd door `Ctrl+Q` je daarentegen **loskoppelen zonder te stoppen**.

PODMAN

Bij Docker betekent "losgekoppeld" dat de daemon de container bijhoudt. Bij Podman is er geen daemon: wanneer `podman run -d` de hand teruggeeft, blijft `common` over — enkele honderden KB, één per container — om de `stdout` / `stderr`-pijpen open te houden, de logs te schrijven en de exitcode te noteren wanneer PID 1 sterft. Sluit je je WSL-sessie, dan sterven `common` en je containers mee... tenzij `systemd` ze vasthoudt (deel 6).

4. Signalen: hoe een container sterft

`podman stop` is geen schakelaar. Het doorloopt een protocol:

1. Verzenden van **SIGTERM** naar PID 1: "stop netjes".
2. Wachten gedurende een **respijtperiode**, standaard 10 seconden (`-t` om ze te wijzigen).
3. Is het proces er nog, dan **SIGKILL**, niet te onderscheppen, onmiddellijk — Podman kondigt het aan: `StopSignal SIGTERM failed to stop container ... in 10 seconds, resorting to SIGKILL`.

`podman kill` springt meteen naar stap 3. En `podman rm -f` doet een volledige `stop`, 10 seconden inbegrepen — vandaar de `-t 0` van lab 01.

→ LINUX

Een **signaal** is een asynchrone melding die de kernel aan een proces bezorgt: **SIGTERM** (15) vraagt om te stoppen en kan onderschept worden, **SIGKILL** (9) doodt zonder beroep, **SIGINT** (2) is `Ctrl+C`. Een programma "behandelt" een signaal door een *handler* te installeren; anders geldt de standaardactie — voor **SIGTERM**: sterven. Behalve voor PID 1, dat geen standaardactie heeft: het negeert.

Tijdens die 10 seconden werkt een goed geschreven Spring Boot-applicatie de lopende verzoeken af, sluit de PostgreSQL-pool, schrijft zich uit bij de service discovery. Met **SIGKILL** niets van dat alles: afgebroken verzoeken, hangende databaseverbindingen, mogelijk inconsistente gegevens.

→ JAVA / SPRING BOOT

De JVM vertaalt **SIGTERM** naar de uitvoering van **shutdown hooks** (`Runtime.addShutdownHook`). Spring Boot registreert er een die de context sluit: `@PreDestroy`, JDBC-pool, webserver. Met `server.shutdown=graceful` aanvaardt de server geen nieuwe verbindingen meer en laat hij de lopende verzoeken afwerken. Dat alles **veronderstelt dat SIGTERM aankomt** bij de JVM.

Twee valkuilen die de ontvangst van het signaal verhinderen:

1. Een shell vóór de applicatie. Het geval van een opstartscript dat de applicatie zonder `exec` start, of van de *shell*-vorm van een `CMD` (`CMD java -jar app.jar` wordt `/bin/sh -c "java -jar app.jar"`). PID 1 is dan `sh`, dat **SIGTERM niet doorgeeft** aan zijn kind: Java krijgt het signaal nooit, wacht 10 seconden en wordt gedood. De remedie is dubbel: `exec`-vorm (`CMD ["java", "-jar", "app.jar"]`) en, in een script, `exec java -jar app.jar` als laatste regel. Dit syntaxdetail (lab 04) bepaalt de kwaliteit van je stops.

2. De bijzondere status van PID 1. Een proces dat **SIGTERM** niet behandelt en als PID 1 draait, is **ongevoelig** voor `podman stop`, en wordt na de wachttijd gedood. PID 1 moet ook de wezen "adopter", anders stapelen *zombies* zich op. Vandaar `--init`, dat een mini-init (`podman-init`) vóór je applicatie plaatst.

5. Exitcodes

```
podman run --rm alpine sh -c 'exit 3'; echo $?      # 3
podman ps -a --format 'table {{.Names}}\t{{.Status}}'
```

De exitcode van de container is die van zijn PID 1, bewaard in zijn status (`Exited (3)`). Enkele codes om te herkennen:

Code	Gebruikelijke betekenis
0	Normale beëindiging
1	Generieke applicatiefout
125	De engine zelf faalde (ongeldige optie)
126	Commando gevonden maar niet uitvoerbaar (of <code>pasta</code> kon de poort niet openen)
127	Commando niet gevonden in de image
137	Gedood door <code>SIGKILL</code> (128+9) — <code>podman kill</code> , einde van de respijtperiode, of de OOM killer
143	Beëindigd door <code>SIGTERM</code> (128+15) — een nette <code>stop</code>

137 zie je in productie het vaakst: een `stop` voorbij de respijtperiode, of een overschreden geheugenlimiet. `podman inspect` beslist: `.State.OOMKilled` is `true` in het tweede geval.

6. De *restart policies* — en wie ze toepast

```
podman run -d --restart=unless-stopped --name api mijn-api:1.0
```

Beleid	Gedrag
<code>no</code> (standaard)	Geen automatische herstart
<code>on-failure[:N]</code>	Herstart als de exitcode $\neq$ 0 is, hoogstens N keer
<code>always</code>	Herstart altijd, ook na een herstart van de host... als er iemand is om het te doen
<code>unless-stopped</code>	Zoals <code>always</code> , behalve als jij hem handmatig gestopt hebt

Bij Docker past de daemon die regels toe, ook bij het opstarten van de machine. Bij Podman herstart `conmon` de container zolang je sessie leeft — na een reboot is er niemand om het beleid te lezen.

PODMAN

Het antwoord van Podman is **systemd**, de dienstbeheerder van Linux, via **Quadlet**: een bestand `~/.config/containers/systemd/api.container` van tien regels (`[Container]`, `Image=`, `PublishPort=...`) en `systemctl --user start api`. De container wordt een gewone dienst: start bij het booten, herstart bij falen, logs in `journalctl`. Daarom wilde Podman geen daemon: die van Linux bestaat al (lab 10).

VALKUIL

`always` herstart een container zelfs na een handmatige `stop`, bij de volgende start van de engine. `unless-stopped` onthoudt je bedoeling: bijna altijd de juiste keuze.

7. Observeren en ingrijpen

```
podman logs -f --tail 50 api      # uitvoerstroom van PID 1
podman exec -it api sh            # nieuw proces IN de container
podman attach api                 # opnieuw aankoppelen op de bestaande PID 1
podman top api                    # processen van de container, gezien vanaf de host
podman stats api                  # CPU/geheugenverbruik in real time
podman inspect api                # volledige toestand, JSON
podman cp api:/app/log.txt .      # een bestand uithalen, zelfs uit een gestopte container
podman events --since 10m         # het logboek van aanmaken, stoppen, sterven
```

`exec` **maakt een nieuw proces aan** in de namespaces van de container — dat wil je om te gaan kijken. `attach` koppelt je opnieuw aan de in-/uitvoer van de **bestaande PID 1**: een `Ctrl+C` stopt daar de container.

`podman logs` toont alleen wat PID 1 naar `stdout` / `stderr` schreef, opgevangen door `common`. Een applicatie die naar een bestand schrijft, verschijnt niet — vandaar de regel: **loggen naar de standaarduitvoer**. Spring Boot doet dat standaard; configureer dus geen `logging.file.name`.

8. In het bedrijf

- De Spring Boot-backend draait met `-d`, met `--restart=unless-stopped` onder Docker of als Quadlet-dienst onder Podman — of zonder beleid onder een orchestrator. Netjes stoppen is een productieonderwerp: `SIGTERM` ontvangen + Spring *graceful shutdown* = uitrollen zonder verloren verzoek.
- De diagnose volgt altijd dezelfde volgorde: `ps -a` (status, code), `logs`, `inspect` (OOM? configuratie?), en dan `exec` als de container nog leeft.

Onthouden

- Een container leeft precies zolang zijn hoofdproces (PID 1). Diensten draaien **op de voorgrond**: geen daemon, `&` of `systemd` in de container.
- `-i` houdt `stdin` open, `-t` kent een terminal toe. `stop` = `SIGTERM`, respijtperiode, dan `SIGKILL` (Podman waarschuwt); `kill` = rechtstreeks `SIGKILL`; `rm -f` = volledige `stop` zonder `-t 0`. De exec-vorm (`["java", "-jar", "x.jar"]`) is onmisbaar om signalen te ontvangen.
- `137` = gedood (KILL of OOM), `143` = netjes gestopt (TERM), `127` = commando niet gevonden.
- `rm` vernietigt de gegevens van de container; `stop` niet. `exec` maakt een proces aan, `attach` koppelt aan PID 1. Zonder daemon gaat herstart bij het booten via `systemd` (Quadlet).

Woordenschat

PID 1: hoofdproces van de container. — **respijtperiode**: tijd tussen `SIGTERM` en `SIGKILL`. — **graceful shutdown**: nette stop die het lopende werk afwerkt. — **restart policy**: regel voor automatische herstart. — **OOM killer**: de kernel doodt een proces als het geheugen op is. — **zombie**: beëindigd proces waarvan niemand de exitcode las. — **common**: toezichhouder van een Podman-container. — **Quadlet**: integratie van Podman in `systemd` (`.container`-bestanden).